# Tamper-Proof Digital Certificate Verification Using Blockchain and QR Authentication


**Presented By:**
KANIGASRI M(192565010)
CHAITHIKA L S(192311092)
BABY SHALINI(192521268)


**Course Code:** CSA0925
**Course Name:** PROGRAMMING IN JAVA


**Guided By:**
Dr.Sivasankar

# PROJECT OVERVIEW

- Digital certificates are widely used to verify academic and professional achievements.
- Traditional paper and PDF certificates can be forged, duplicated, or modified.
- The proposed system provides a **Java-based certificate generation and verification platform**.
- **SHA-256 hashing** is used to detect unauthorized modifications.
- **QR codes** provide quick and convenient certificate verification.
- **Blockchain concepts** are used to maintain secure certificate records.
- The system is designed to provide a simple, reliable, and tamper-resistant verification process.

# Problem Statement & Objectives

**Problem Statement**

- Fake certificates are increasing.
- Manual verification is slow.
- Certificates can be easily modified.
- No reliable verification mechanism.

**Objectives**

- Prevent certificate forgery.
- Secure certificate records using blockchain.
- Enable instant QR code verification.
- Detect certificate tampering.
- Maintain transparent audit records.

# MOTIVATION & NEED

- Growing use of **digital certificates** in education and professional environments.
- Increasing risk of **certificate forgery and unauthorized modification**.
- Manual verification requires significant **time and effort**.
- A secure system is needed to verify certificates **quickly and reliably**.
- **QR authentication** enables instant verification using a smartphone.
- **Hashing and blockchain** improve data integrity and trust.
- The system reduces dependency on manual verification by providing an **automated authentication process**.

# Existing System / Current Practice

**Traditional Certificate Verification**

1. Certificate is submitted by the candidate.
2. Institution manually checks the certificate.
3. Details are compared with institutional records.
4. Verification is completed manually.
5. There is limited automated tamper detection.

**Limitations**

- Time-consuming
- Manual verification
- Difficult to verify large numbers of certificates
- Vulnerable to forged or modified documents
- Limited real-time verification
- No convenient verification mechanism for external users

# RESEARCH / KNOWLEDGE GAP

| Existing Approach | Gap |
|---|---|
| Paper-based certificates | Can be physically forged |
| PDF certificates | Can be copied or modified |
| Manual verification | Slow and requires human effort |
| QR-only verification | QR itself does not guarantee data integrity |
| Centralized records | Greater dependency on a single authority |
| Hash-based verification alone | Does not provide a complete audit mechanism |

# PROPOSED SOLUTION

- Develop a **Java-based digital certificate verification system**.
- Generate certificates with a unique **SHA-256 hash**.
- Embed a **QR code** into every certificate.
- Store certificate hash information using **blockchain concepts**.
- Allow anyone to scan the QR code and verify the certificate instantly.
- Compare certificate information against the secured blockchain record.
- Detect unauthorized modifications and maintain an **audit trail**.

The project specification specifically defines the three modules as certificate issuance, QR-based instant verification, and tamper detection/audit.

# Requirements & Specifications

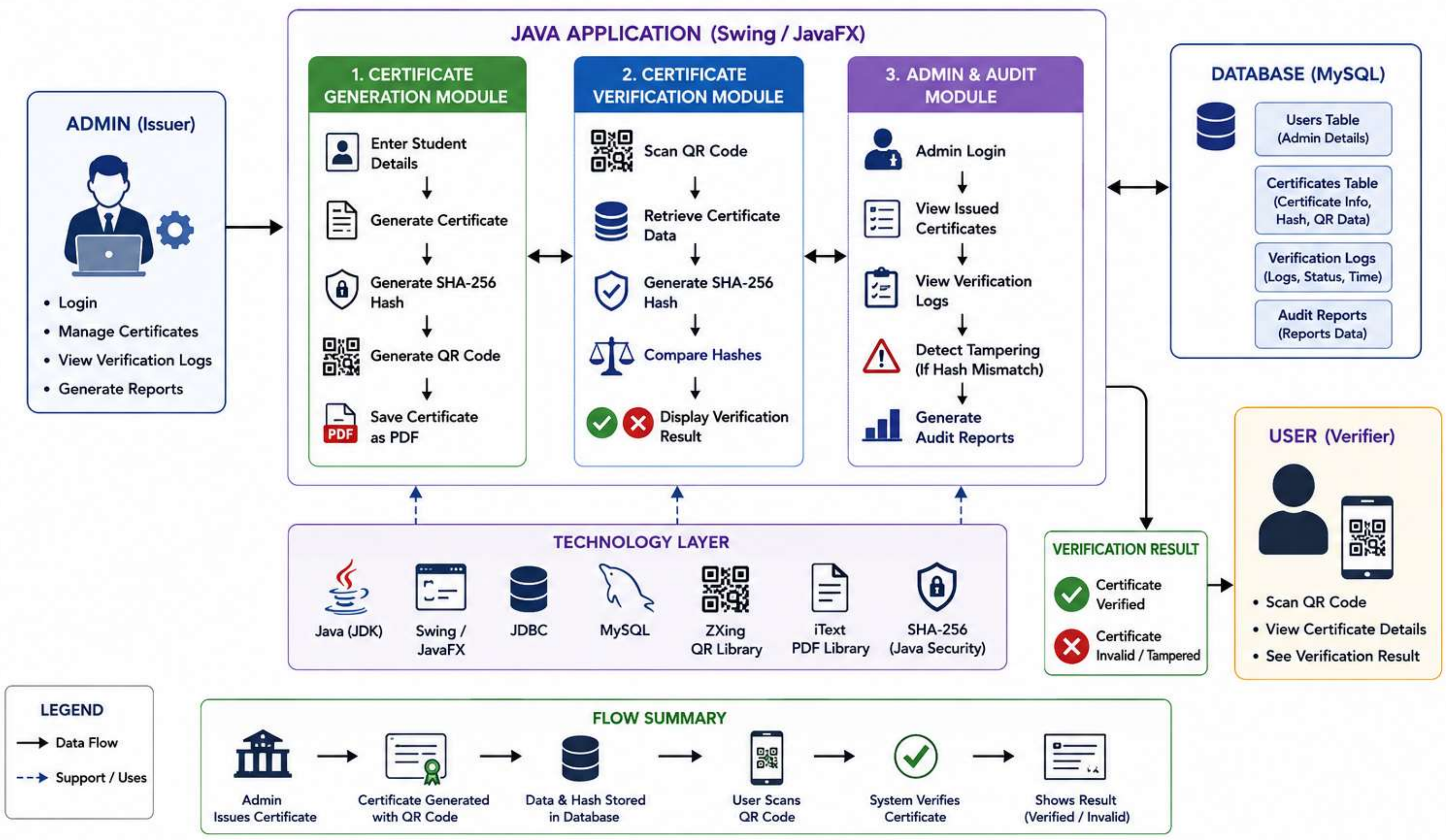
**FUNCTIONAL REQUIREMENTS**

- Administrator login
- Certificate generation
- Unique certificate ID generation
- SHA-256 hash generation
- QR code generation
- Certificate storage
- QR-based certificate verification
- Authenticity checking
- Tamper detection
- Audit log generation

**NON-FUNCTIONAL REQUIREMENTS**

- **Security:** Certificate data must be protected from unauthorized modification.
- **Reliability:** Verification should produce consistent results.
- **Performance:** Verification should be fast.
- **Usability:** QR verification should be simple for users.
- **Scalability:** System should support increasing numbers of certificates.
- **Maintainability:** Java-based modular design should allow future enhancements.

# System Architecture



# Workflow Steps

1. Administrator logs into the system.
2. Student certificate details are entered.
3. A unique certificate hash is generated.
4. A QR code is embedded in the certificate.
5. Certificate information and hash are stored securely.
6. User scans the QR code.
7. The system retrieves the certificate record.
8. Hash values are compared.
9. The certificate is marked **AUTHENTIC** or **TAMPERED**.
10. The verification event is added to the audit trail.

# TECHNOLOGY STACK

| Technology | Purpose |
|---|---|
| Java | Core application development |
| JSP / HTML | Web interface |
| CSS | User interface styling |
| Servlets | Backend request processing |
| MySQL | Database management |
| JDBC | Java–MySQL connectivity |
| SHA-256 | Hash generation and tamper detection |
| QR Code | Instant certificate verification |
| Blockchain | Secure hash-based certificate record |
| Apache Tomcat | Web application server |
| VS Code | Development environment |

# Module 1 – Certificate Generation

**Description**

This module allows the administrator to create digital certificates using student information. The application generates a unique SHA-256 hash and embeds a QR code into the certificate.

**Process**

- Enter student details
- Generate certificate
- Create SHA-256 hash
- Generate QR code
- Save certificate as PDF

**Output**

- Digital certificate
- QR-enabled certificate

# Module 2 – Certificate Verification

**Description**

Users scan the QR code to verify certificate authenticity. The Java application compares the stored hash with the current certificate hash.

**Process**

- Scan QR code
- Retrieve certificate data
- Compare SHA-256 hashes
- Validate certificate
- Display verification status

**Output**

- Certificate Verified
- Certificate Invalid
- Verification details

# Module 3 – Admin & Audit Module

**Description**

The administrator manages certificate records and monitors verification history. The system records every verification attempt for auditing.

**Process**

- Login as administrator
- View issued certificates
- Monitor verification logs
- Detect modifications
- Generate audit reports

**Output**

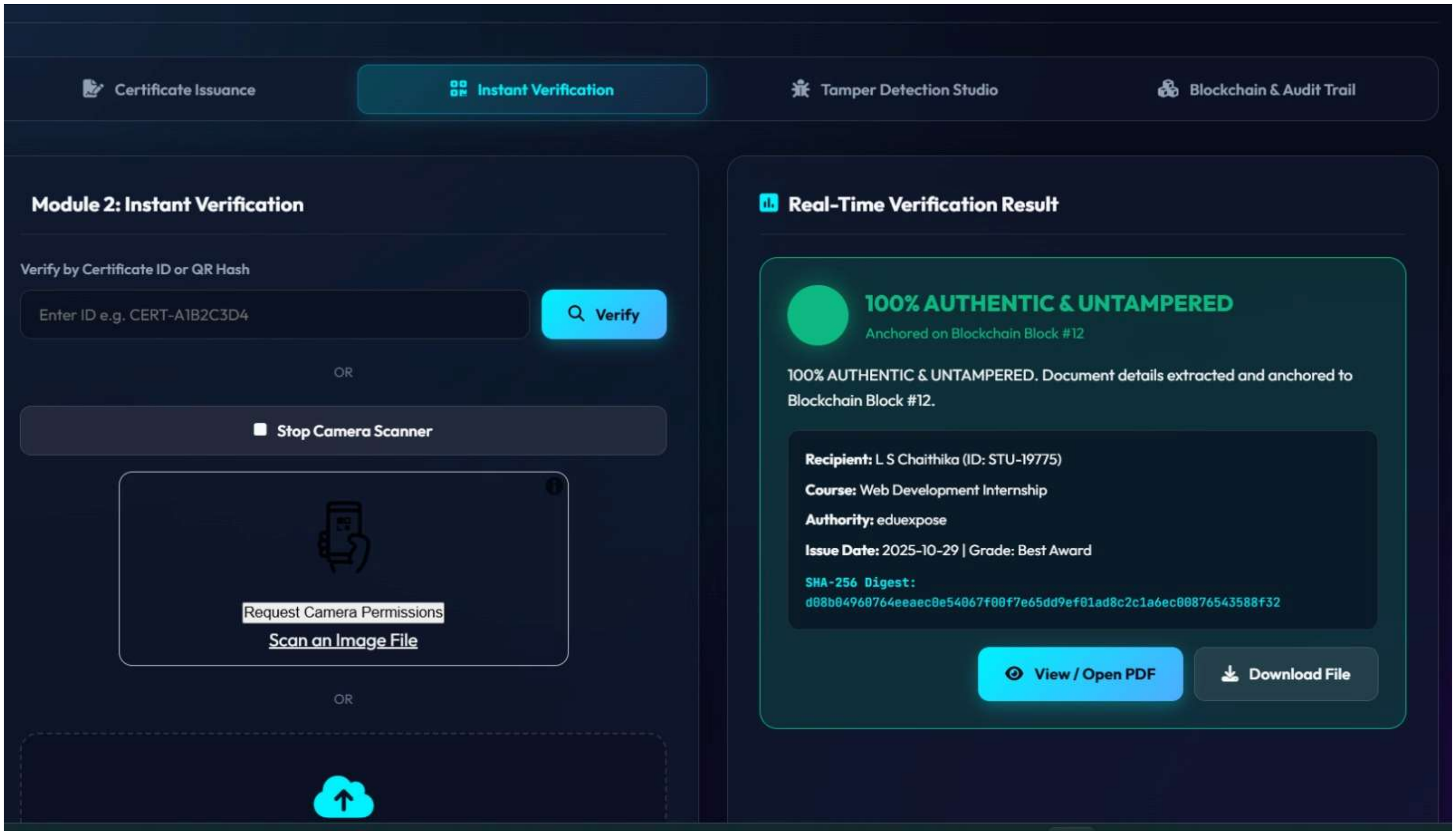
- Audit logs
- Verification history
- Tampering alerts

# Key Algorithm / Technical Implementation

**Hash Generation**
Student Details
↓
Certificate ID
↓
Combine Data
↓
SHA-256 Algorithm
↓
Unique Hash
↓
Store Hash

**Verification Algorithm**
Certificate Data
↓
Generate Current Hash
↓
Compare with Stored Hash
↓
Hash Match?
YES | NO
↓ ↓
AUTHENTIC | TAMPERED

# Prototype / Working Model



# Testing & Validation

| Test Case | Expected Result |
|---|---|
| Admin login with valid credentials | Login successful |
| Admin login with invalid credentials | Access denied |
| Enter valid student details | Certificate generated |
| Generate certificate | QR code and hash created |
| Scan valid QR code | Certificate details displayed |
| Verify unchanged certificate | AUTHENTIC |
| Modify certificate information | TAMPERED |
| Verify unknown Certificate ID | Certificate not found |
| Check audit log | Verification event recorded |

## Comparison with Existing Solutions
## EXISTING vs PROPOSED SYSTEM

| Feature | Existing System | Proposed System |
|---|---|---|
| Certificate Generation | Manual / conventional | Digital |
| Verification | Manual | QR-based |
| Tamper Detection | Limited | SHA-256 based |
| Certificate Integrity | Conventional storage | Hash-based record |
| Verification Speed | Slow | Faster |
| Audit Trail | Limited | Automated |
| Certificate Tracking | Difficult | Certificate ID |
| Accessibility | Institution-dependent | QR-based access |

# LIMITATIONS

- Requires internet/network access for online verification.
- Depends on availability of the verification server.
- Current implementation is primarily designed for educational demonstration.
- Blockchain functionality is implemented as a **hash-based integrity concept**, rather than a large distributed blockchain network.
- QR verification depends on a valid QR code and certificate record.
- Database security and large-scale deployment require additional measures.

## FUTURE ENHANCEMENTS

- ☁️ **Cloud deployment** for large-scale access.
- 📱 **Mobile application** for certificate verification.
- 🔗 Integration with a **real blockchain network**.
- 🔐 Stronger administrator authentication.
- 📊 Advanced analytics for verification activity.
- 🌐 Multi-institution certificate verification.
- 📁 Support for additional certificate formats.
- 🔔 Automated alerts for suspicious verification attempts.

# Conclusion

- The proposed Java application provides a secure and efficient method for generating and verifying digital certificates.
- By combining **Java, MySQL, JDBC, SHA-256 hashing, and QR code technology**, the system minimizes certificate forgery and simplifies the verification process.
- It offers faster authentication, improved security, and easy certificate management.
- The project demonstrates the practical application of Java programming in solving real-world security challenges and can be further enhanced with cloud integration and mobile-based verification.

# References (2021–2025)


- NIST — **Secure Hash Standard (SHA-256)**
- Oracle Java Documentation — **MessageDigest / Java Security**
- MySQL Documentation — **MySQL Database**
- JDBC Documentation — **Java Database Connectivity**
- ZXing Documentation — **QR Code Generation and Decoding**
- Apache Tomcat Documentation
- Relevant research papers on **blockchain-based academic certificate verification**
- Project requirement and capstone documentation

# THANK YOU!!